



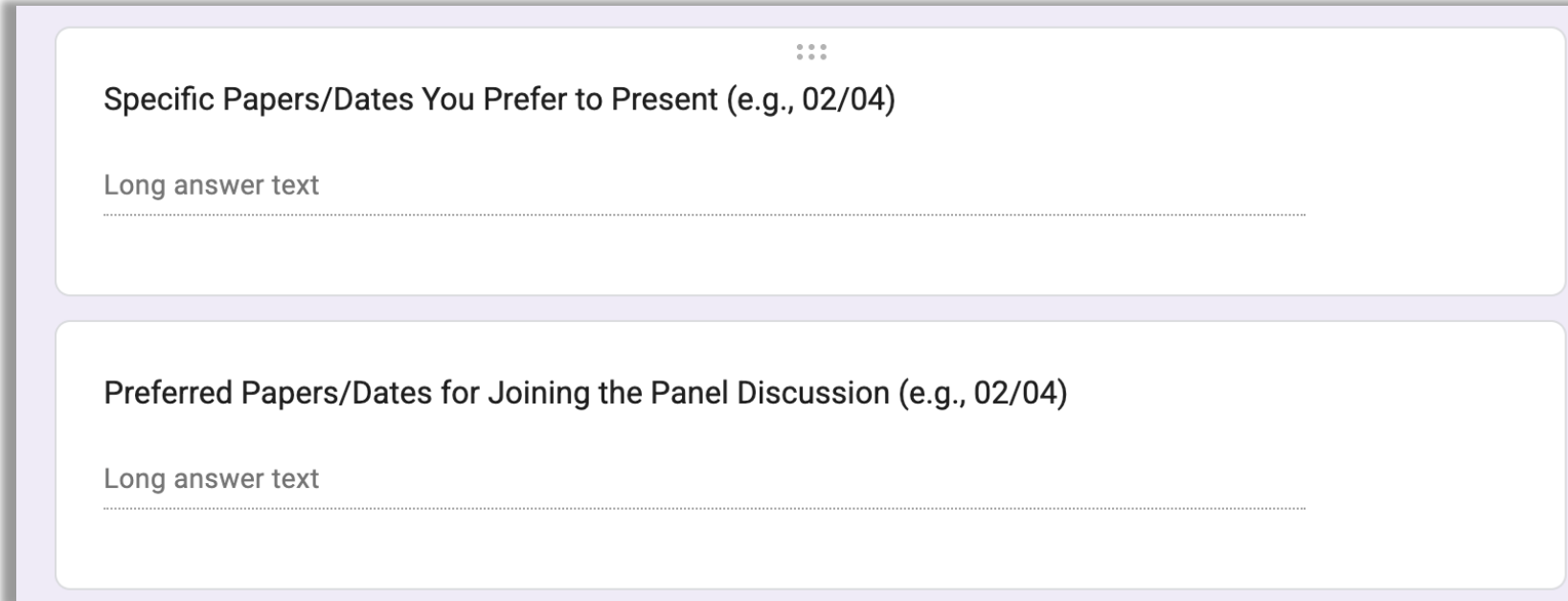
CS 598: Transformers Deep Dive

Fan Lai

The Grainger College of Engineering

Materials with thanks to Minjia Zhang, Arvind Krishnamurthy,
and many colleagues

- **Team registration**
 - **Five** members in a team by Feb 1st



The image shows a screenshot of a registration form with two distinct sections, each with a title, a three-dot menu icon, and a long-answer text field.

Section 1:

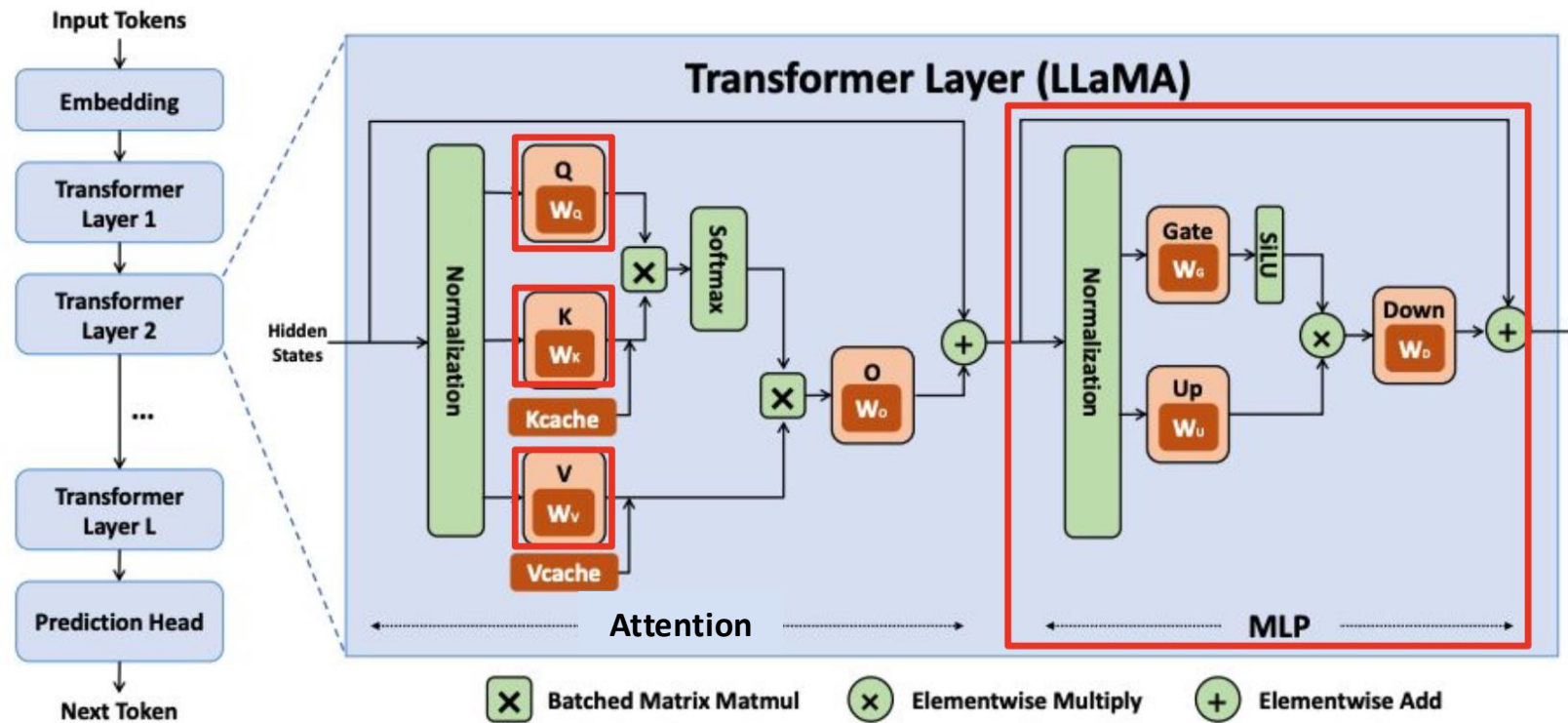
- Menu icon: ⋮
- Title: Specific Papers/Dates You Prefer to Present (e.g., 02/04)
- Text field: Long answer text

Section 2:

- Menu icon: ⋮
- Title: Preferred Papers/Dates for Joining the Panel Discussion (e.g., 02/04)
- Text field: Long answer text

- **No class this Friday (for proposal Q&A in SC 3128)**

Recall: Self-Attention in Transformers

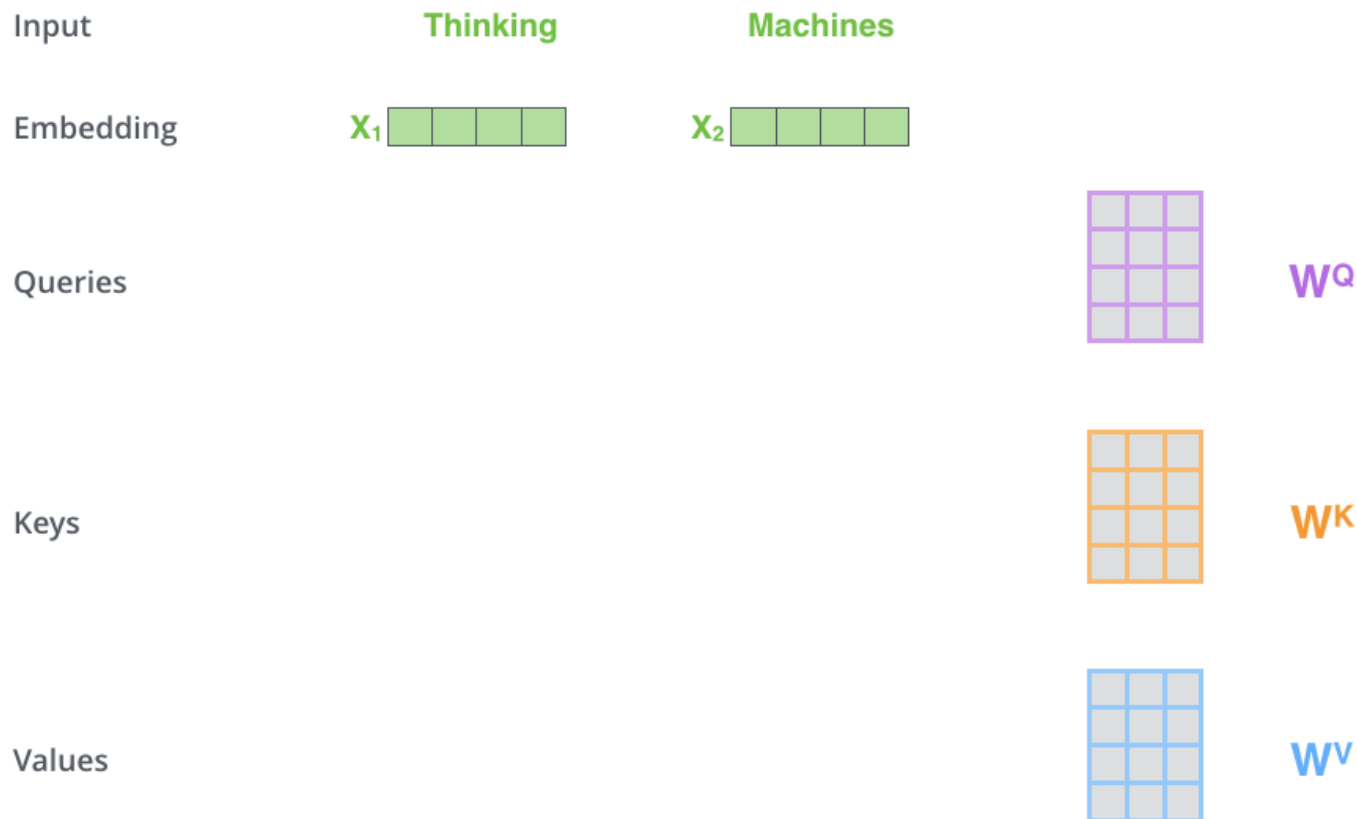


Recall: Self-Attention in Transformers



- **Step 1:** compute “key, value, query” embedding for each input token

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

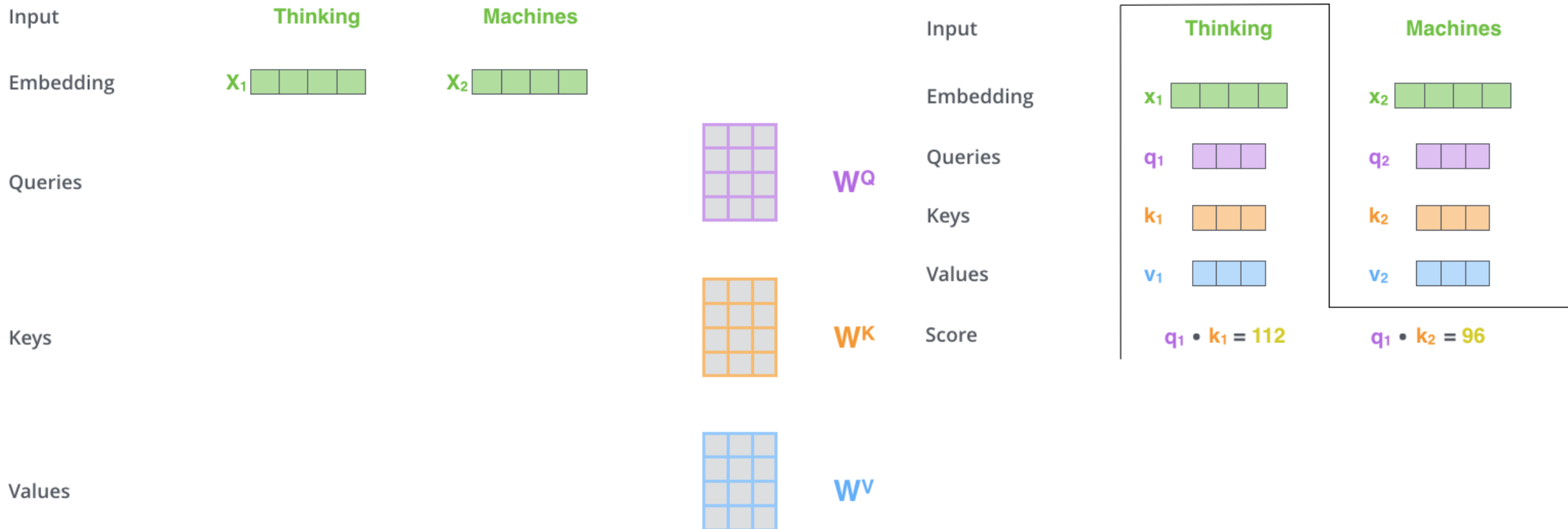


Recall: Self-Attention in Transformers



- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

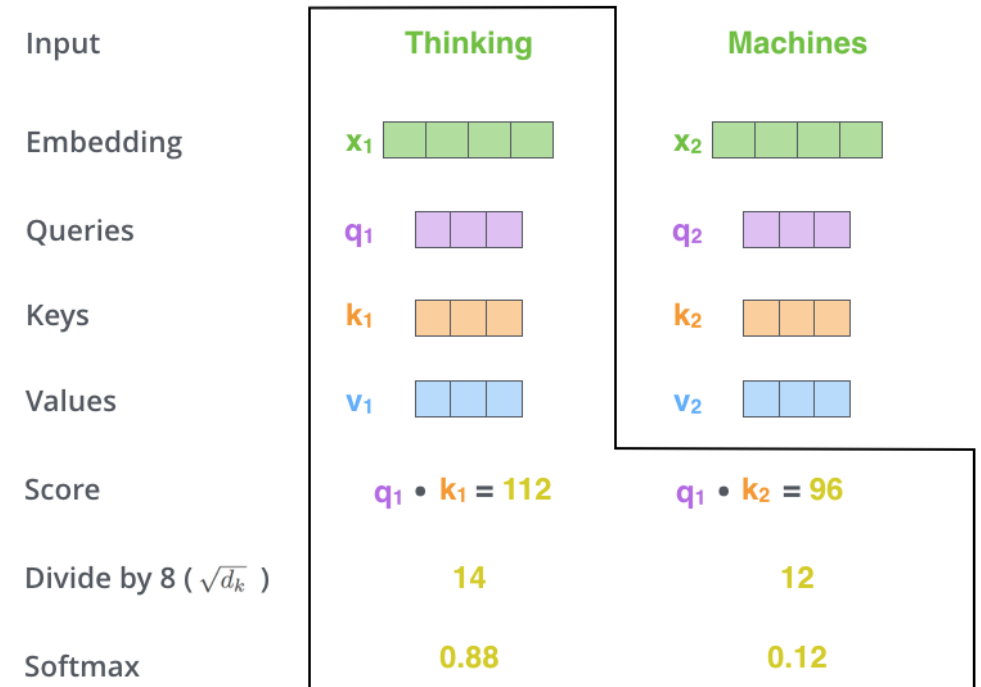
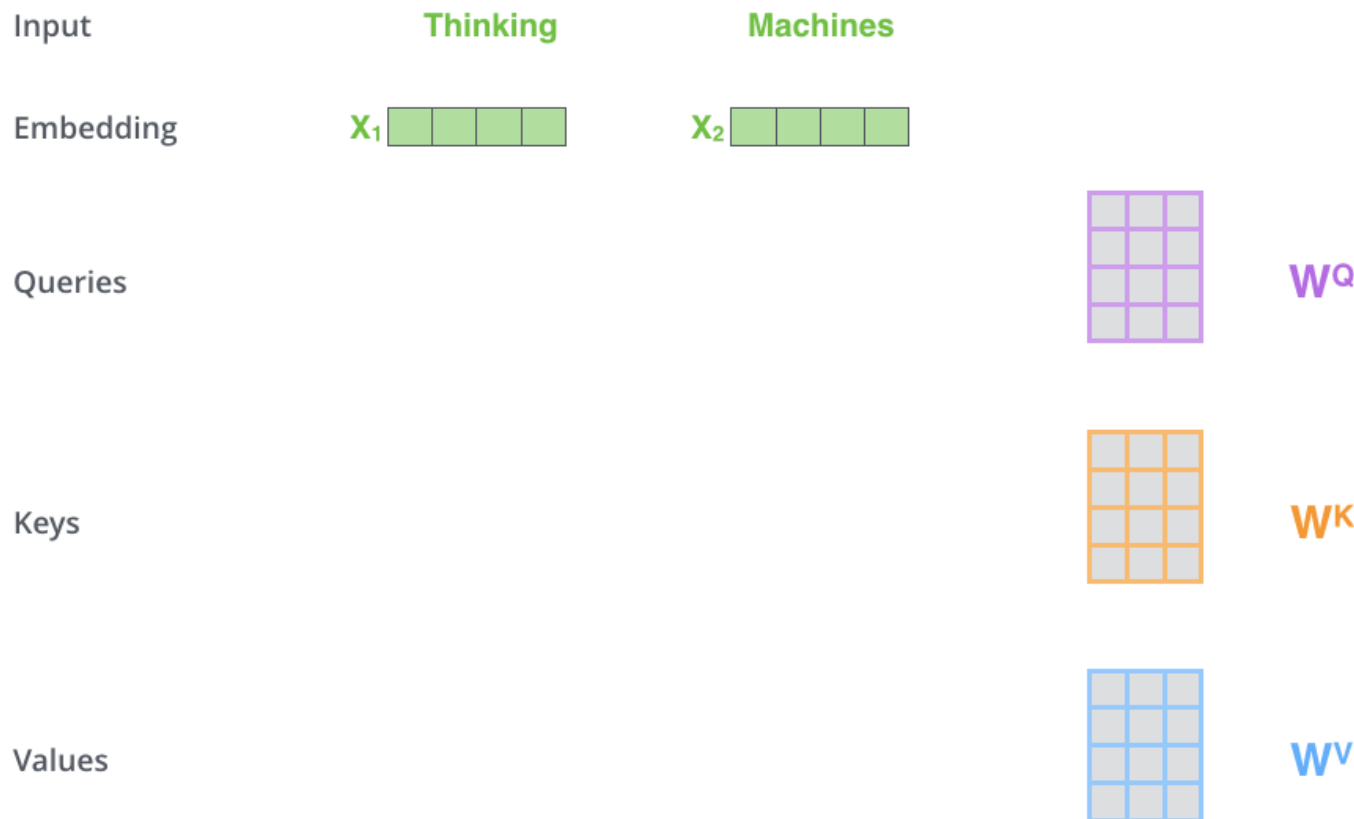


Recall: Self-Attention in Transformers



- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens
- **Step 3:** compute normalized attention scores

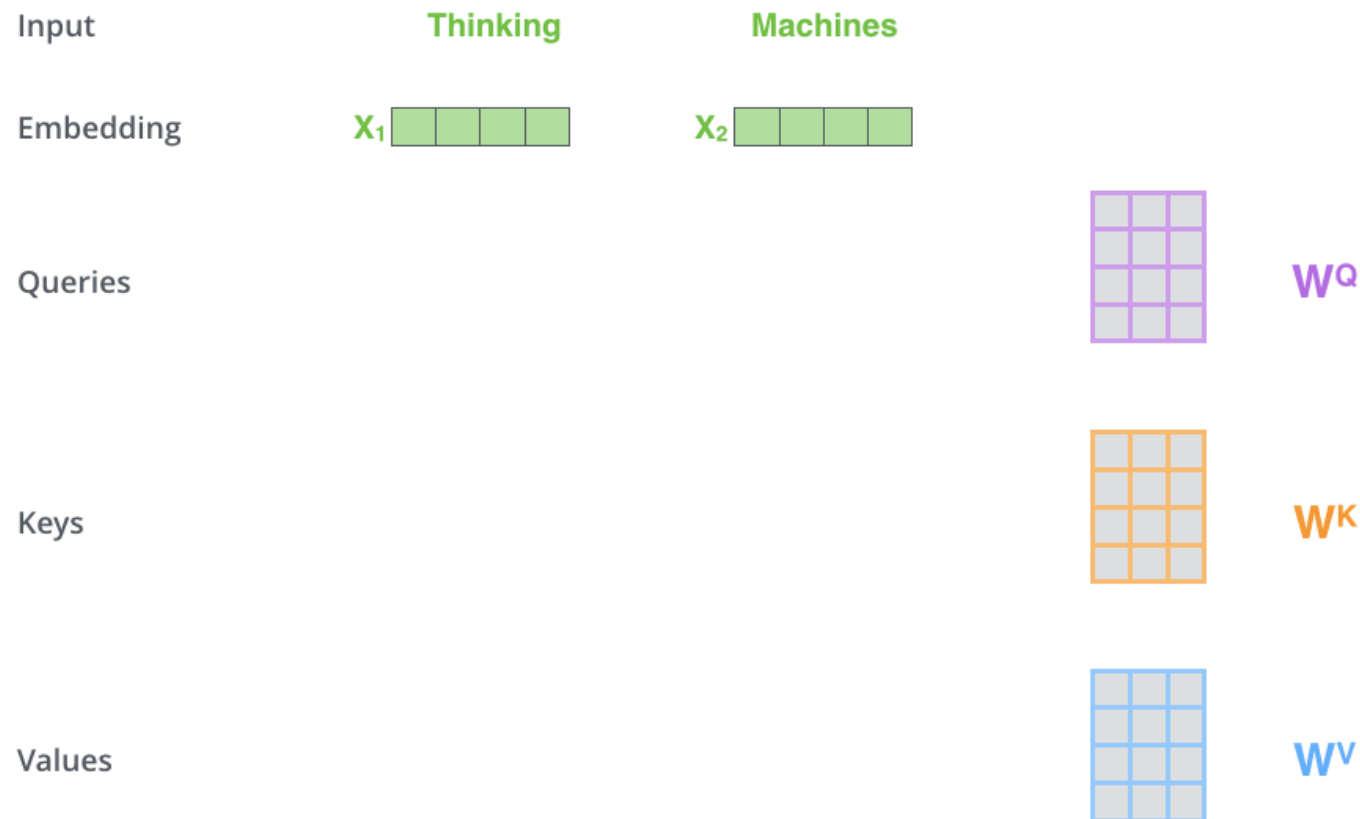
$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$



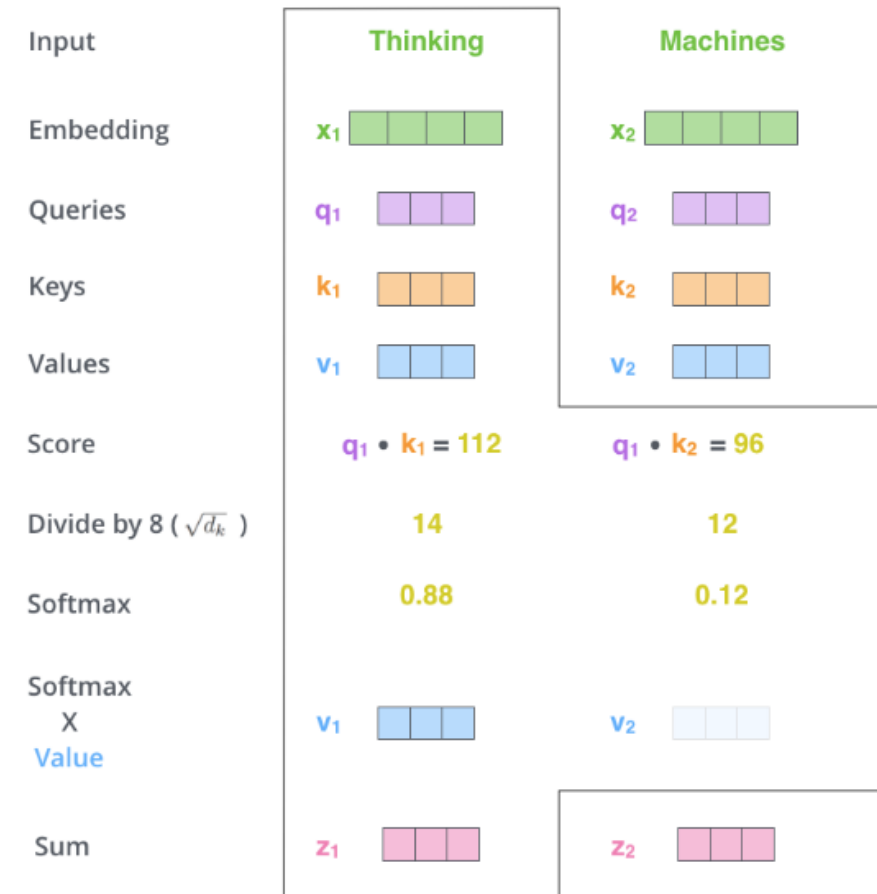
Recall: Self-Attention in Transformers



- **Step 1:** compute “key, value, query” embedding for each input token
- **Step 2:** compute scores between pairs of tokens
- **Step 3:** compute normalized attention scores
- **Step 4:** get new representation by weighted sum of values



$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

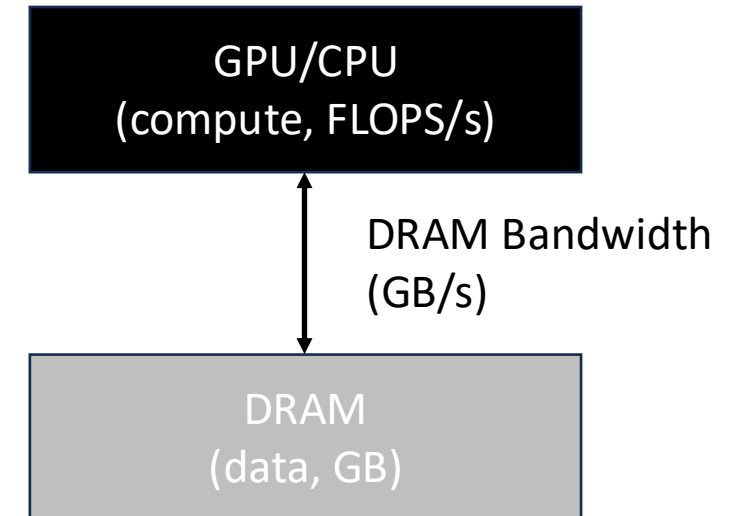


- **Transformers Deep Dive**

- Performance Analysis (e.g., Arithmetic Intensity)
- Multi-head & Multi-query Attention
- Native Sparse Attention (ACL'25 Best Paper Award)

Arithmetic Intensity (AI): evaluate the efficiency of computational algorithms

- $AI = \#ops / \#bytes$



Arithmetic Intensity (AI): evaluate the efficiency of computational algorithms

- $AI = \text{\#ops} / \text{\#bytes}$

System performance is bound by

- The peak compute FLOPS (Floating-Point Ops)
 - The memory bandwidth
-
- Use intensity analysis to guide system optimization and hardware development

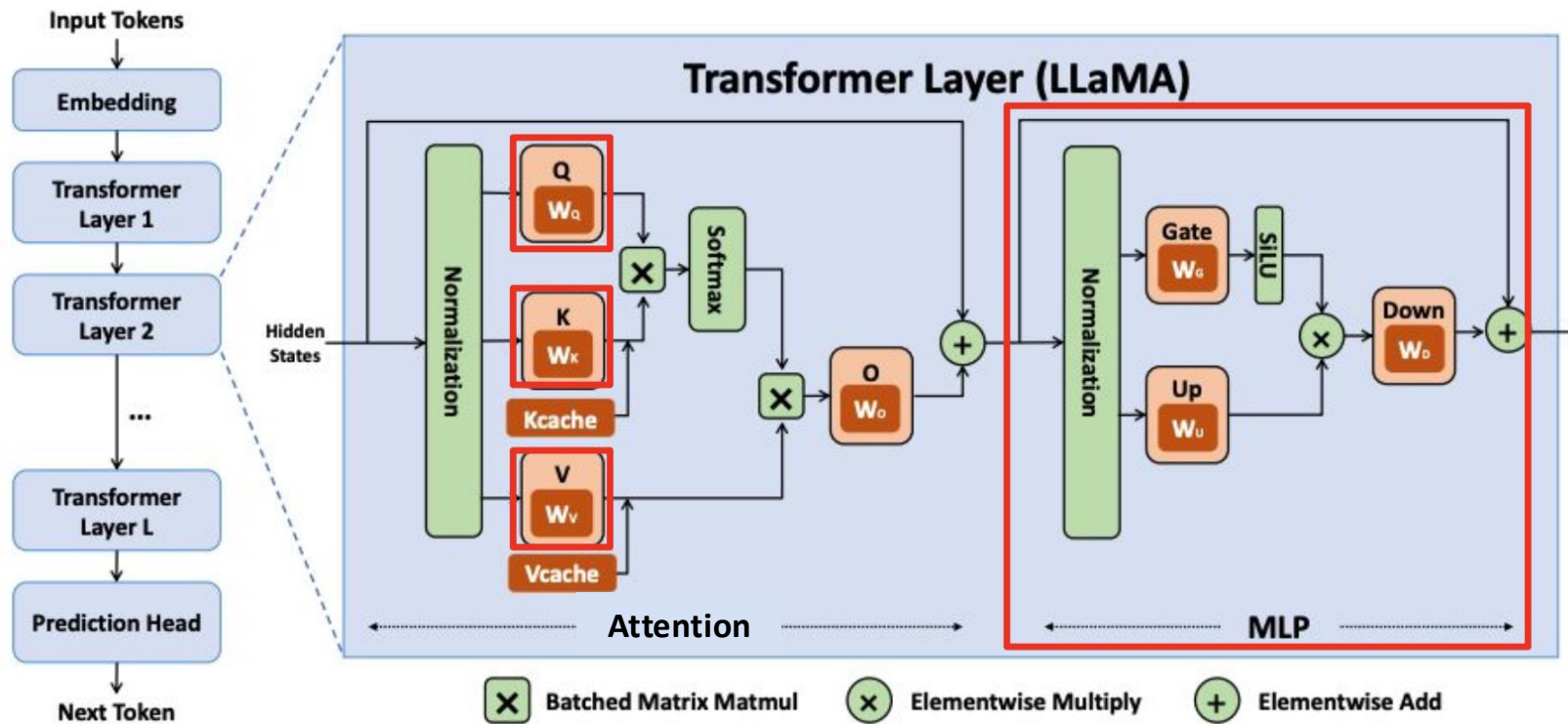
```
void add(int n, float* A, float* B, float* C){  
    for (int i=0; i<n; i++)  
        C[i] = A[i] + B[i];  
}
```

Two loads, one store per math op

1. Read A[i]
2. Read B[i]
3. Add A[i] + B[i]
4. Store C[i]

Arithmetic intensity = 1/3

Arithmetic Intensity: Transformers



Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- Step 1: $\mathbf{Q}$ matrix $(N \times h)$ x $\mathbf{K}^T$ matrix $(h \times N)$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

A

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

B

1	2	3	4
5	6	7	8
9	10	11	12
13	14	15	16

Compute Flops:

- Step 1: Q matrix $(N * h)$ x K^T matrix $(h * N)$

$$C_{i,j} = A_{(i,)} \times B_{(:,j)} : \underbrace{1 \times 1 + 2 \times 5 + 3 \times 9 + 4 \times 13}_{\text{h-element mul \& add} \sim 2 * h \text{ Flops}}$$

$$C = A \times B$$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

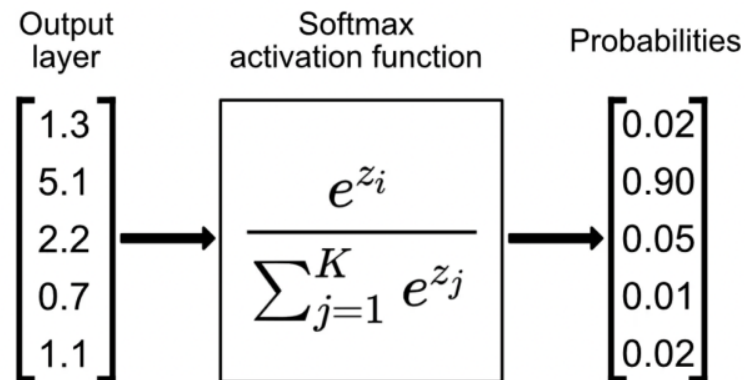
- Step 1: $\underbrace{Q \text{ matrix}_{(N \times h)} \times K^T \text{ matrix}_{(h \times N)}}_{\text{Flops: } 2 * h * N * N}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-



Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Loads/stores: $\underbrace{(2 * h * N + N * N)}_{\text{Step 1}} * 2$ (due to mul/add ops)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Loads/stores: $\underbrace{2 * 2 * h * N + 2 * N * N}_{\text{Step 1}} + 3 * N * N + N * N + 2(N * N + N * h) + 2 * N * h$

Let us start with the attention block: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$

Algorithm 0 Standard Attention Implementation

Require: Matrices $\mathbf{Q}, \mathbf{K}, \mathbf{V} \in \mathbb{R}^{N \times d}$ in HBM.

- 1: Load $\mathbf{Q}, \mathbf{K}$ by blocks from HBM, compute $\mathbf{S} = \mathbf{QK}^T$, write $\mathbf{S}$ to HBM.
 - 2: Read $\mathbf{S}$ from HBM, compute $\mathbf{P} = \text{softmax}(\mathbf{S})$, write $\mathbf{P}$ to HBM.
 - 3: Load $\mathbf{P}$ and $\mathbf{V}$ by blocks from HBM, compute $\mathbf{O} = \mathbf{PV}$, write $\mathbf{O}$ to HBM.
 - 4: Return $\mathbf{O}$.
-

Notations:

- h : Hidden dimension of QKV (often 4096)
- N : Input length (e.g., 4096 tokens)

Compute Flops:

- $\underbrace{2 * h * N * N}_{\text{Step 1 (matrix mul)}} + \underbrace{3 * N * N}_{\text{Step 2 (softmax)}} + \underbrace{2 * N * N * h}_{\text{Step 3}}$

Load: $8 * N * h$; Compute: $(4 * h + 3) * N^2$; Memory IO: $8 * N^2 + 8 * N * h$

Step 1

Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot h$

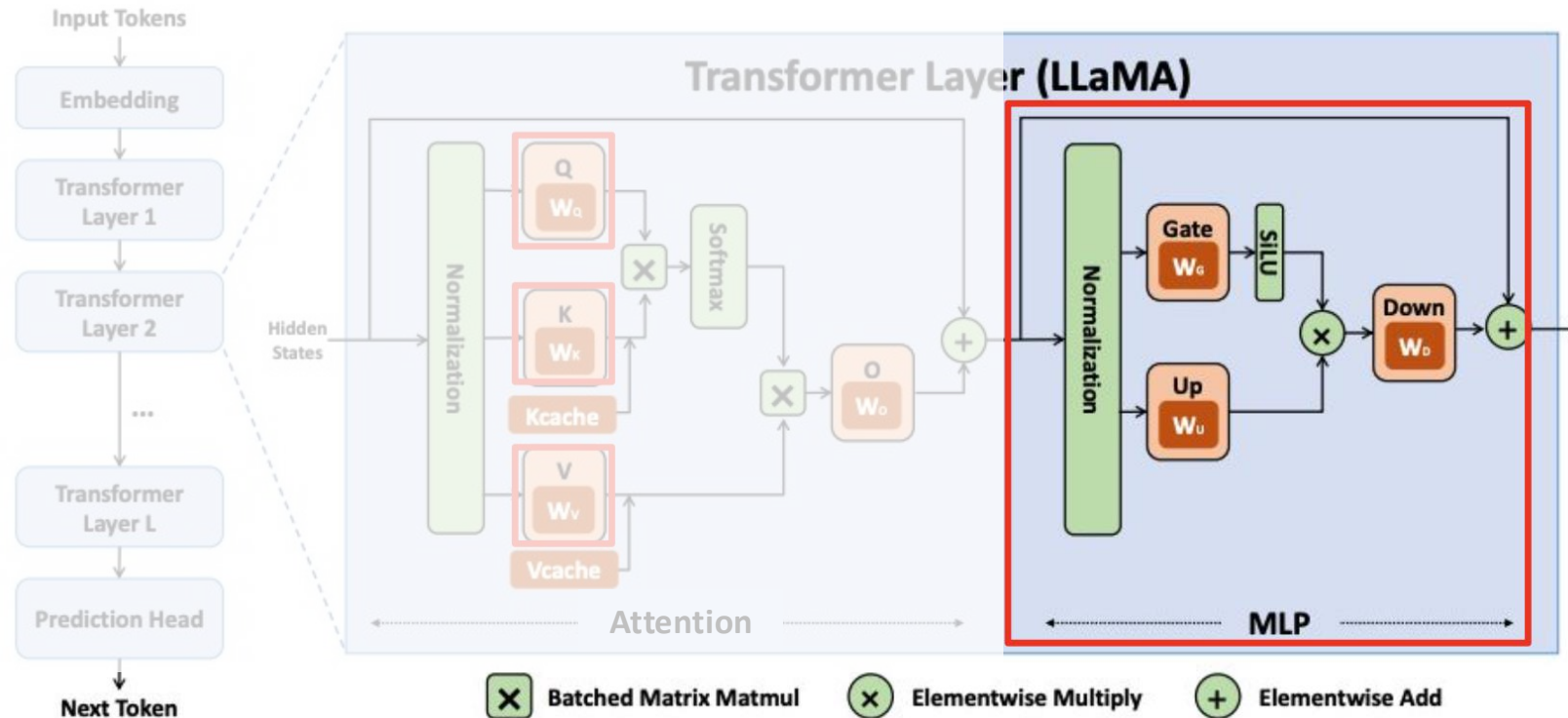


$$\text{Arithmetic Intensity} = \{(4 \cdot h + 3) \cdot N^2\} / \{8 \cdot N^2 + 8 \cdot N \cdot h\}$$



Arithmetic intensity increases as the input length N grows.

Arithmetic Intensity: Transformers (MLP)



Linear layer, essentially a GEMM: X
 $\ast W$

- Shape: $X(N, K), W(K, M)$

Compute: $2 \ast N \ast K \ast M$

Memory: $2 \ast N \ast K + 2 \ast K \ast M + 2 \ast N \ast M$

MLP FLOPs/Memory ops = 1365 ops/byte

- $N = K = M = 4096$

A10 GPU Analysis:

- Compute capability: 125TF, Memory bandwidth: 600GB/s
- Ops/byte = $125\text{TF} / 600\text{GB/s} = 208.3 \text{ ops/byte}$

MLP FLOPs/Memory ops = **1365** ops/byte

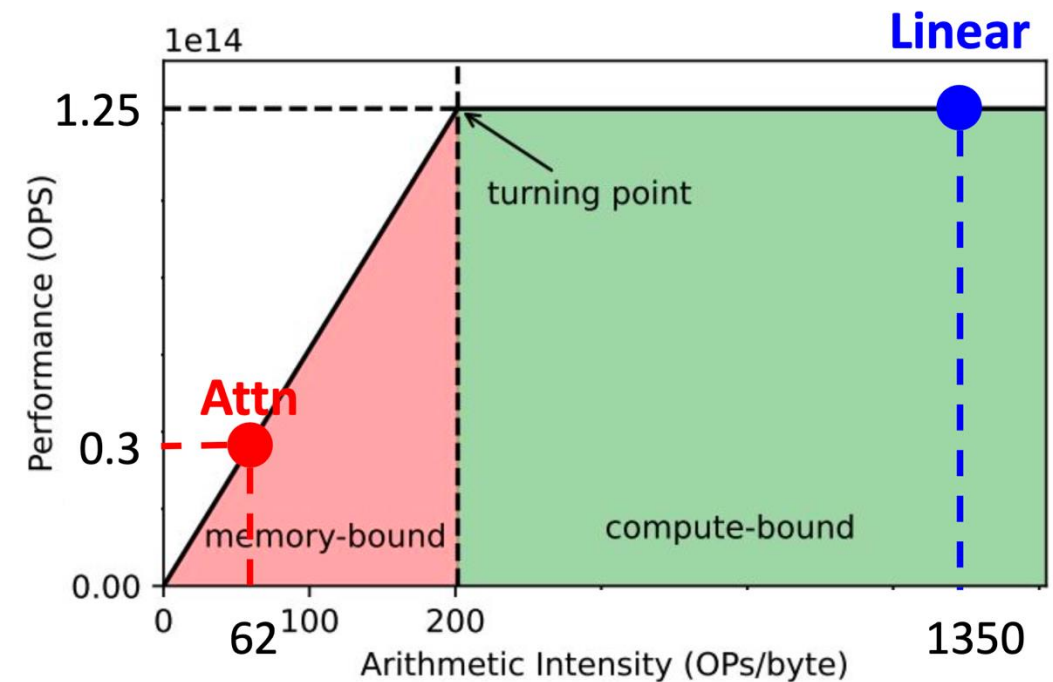
- $N = K = M = 4096$

A10 GPU Analysis:

- Compute capability: 125TF, Memory bandwidth 600GB/s
- Ops/byte = $125\text{TF} / 600\text{GB/s} = \mathbf{208.3}$ ops/byte

MLP's AI is much higher than ~200 ops/byte

- Compute bound



Short input length (N)

How to handle the discrepancy of these two transformer components to maximize efficiency?

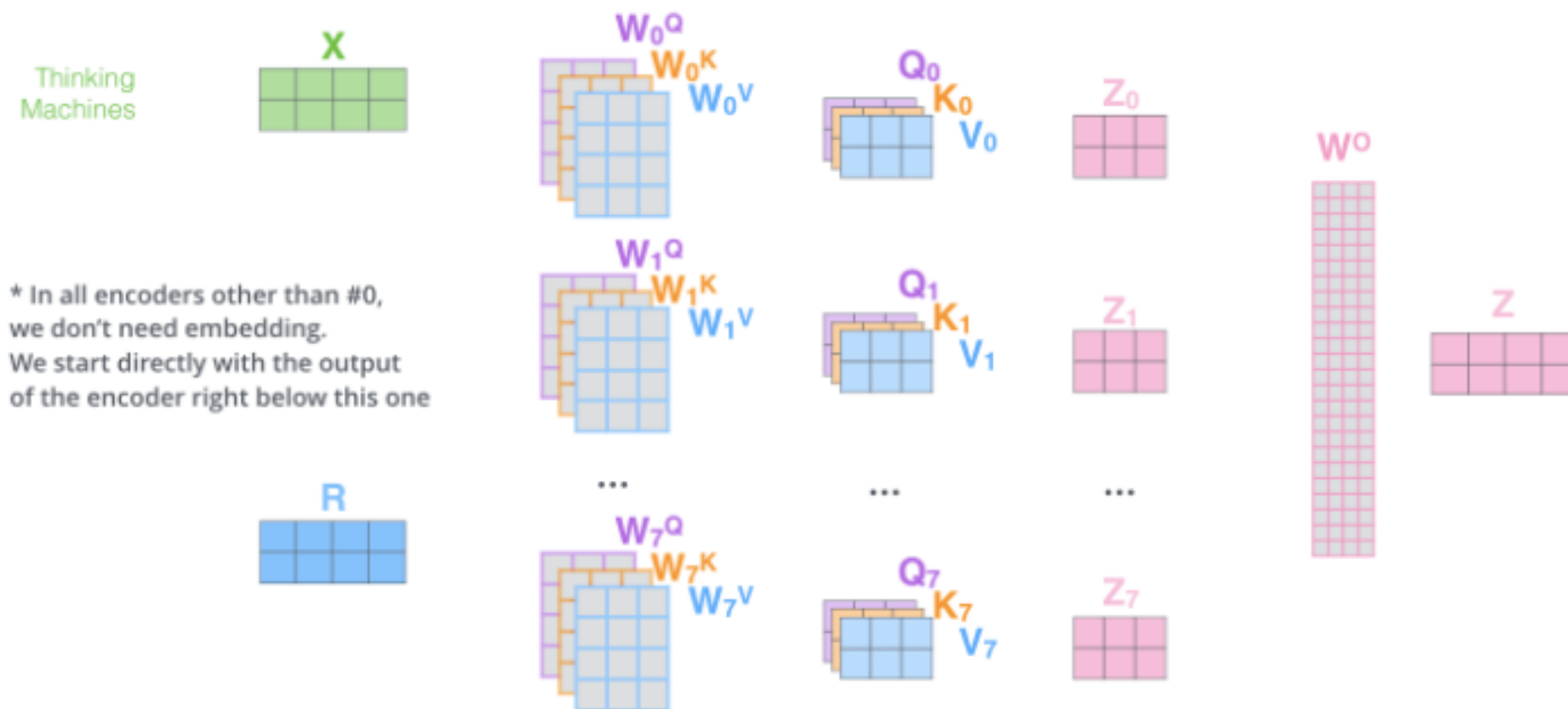
- **Transformers Deep Dive**

- Performance Analysis (e.g., Arithmetic Intensity)
- Multi-head & Multi-query Attention
- Native Sparse Attention (ACL'25 Best Paper Award)

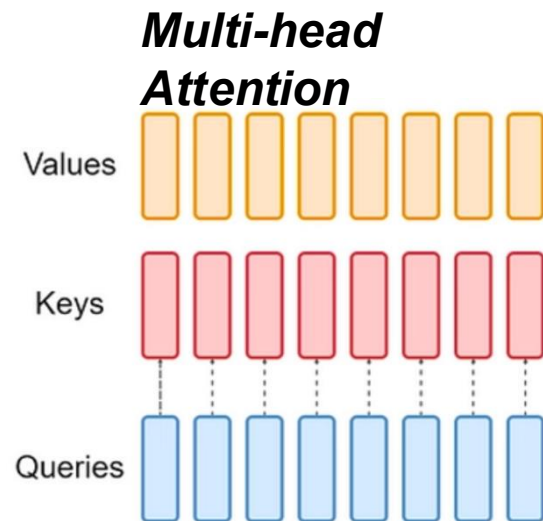
Multi-head Attention



- Do many attention head calculation *in parallel, and combine*
- Each head has its *own* set of parameters

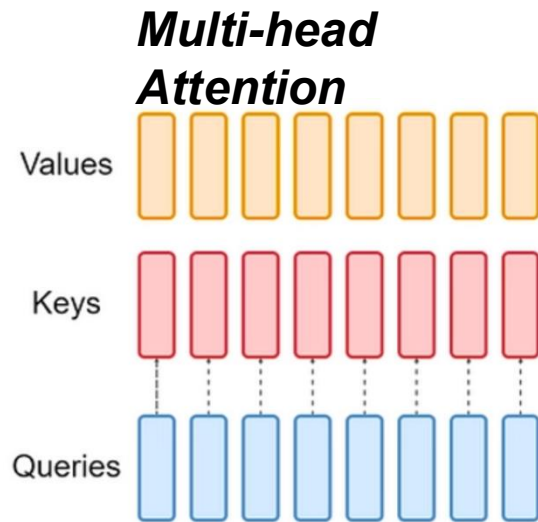


- Do many attention head calculation *in parallel, and combine*
- Each head has its *own* set of parameters
- Different heads can learn different “interactions” between inputs

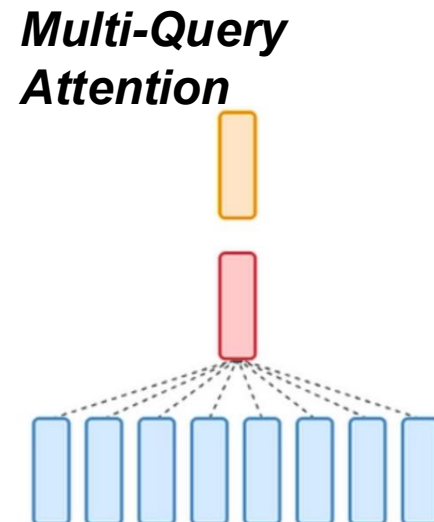


*Each head digests QKV
separately*

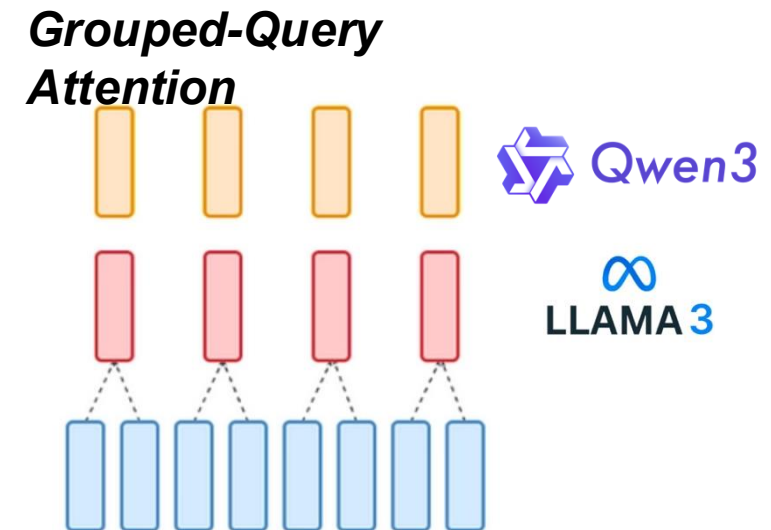
Parallel execution is good, but
memory footprint is still large



Each head digests QKV separately



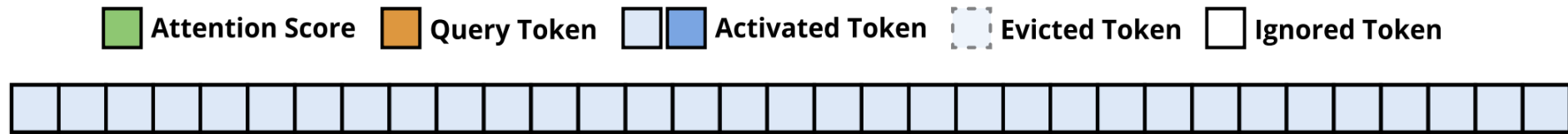
Share single key and value heads across query heads



*Share single key and value heads for **each group** of query heads*

Better compute, smaller memory footprint, but quality-efficiency tradeoff

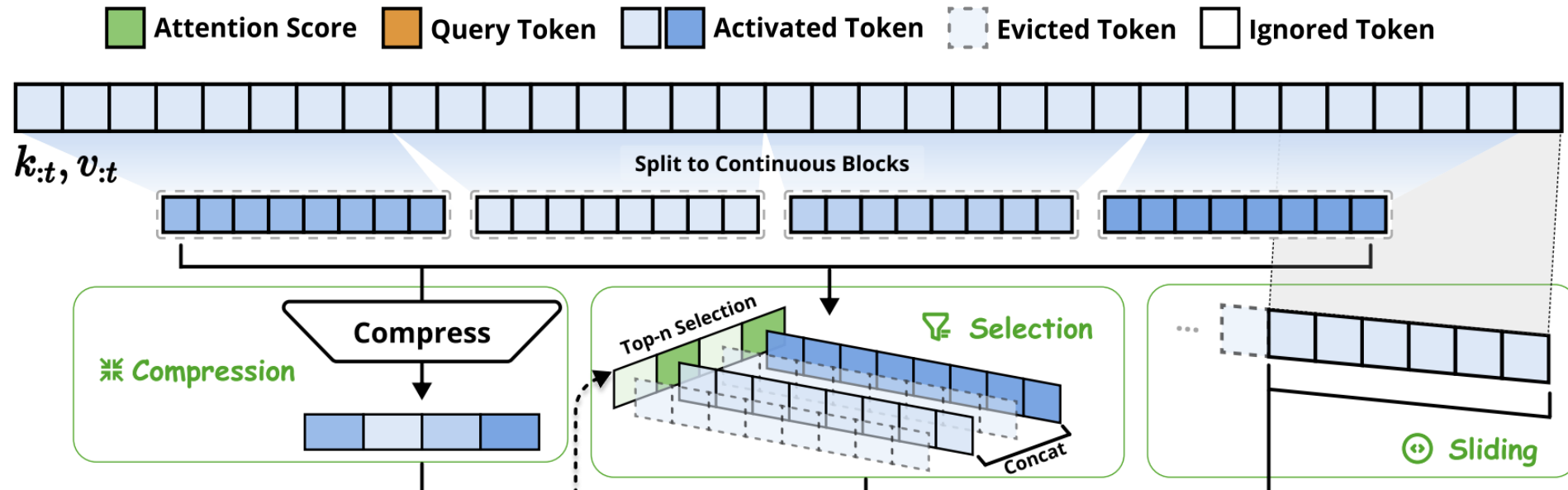
- Key insight: we can summarize long-context input, identify and focus on key words



Native Sparse Attention (ACL'25 from DeepSeek)



- Key insight: we can summarize long-context input, identify and focus on key words

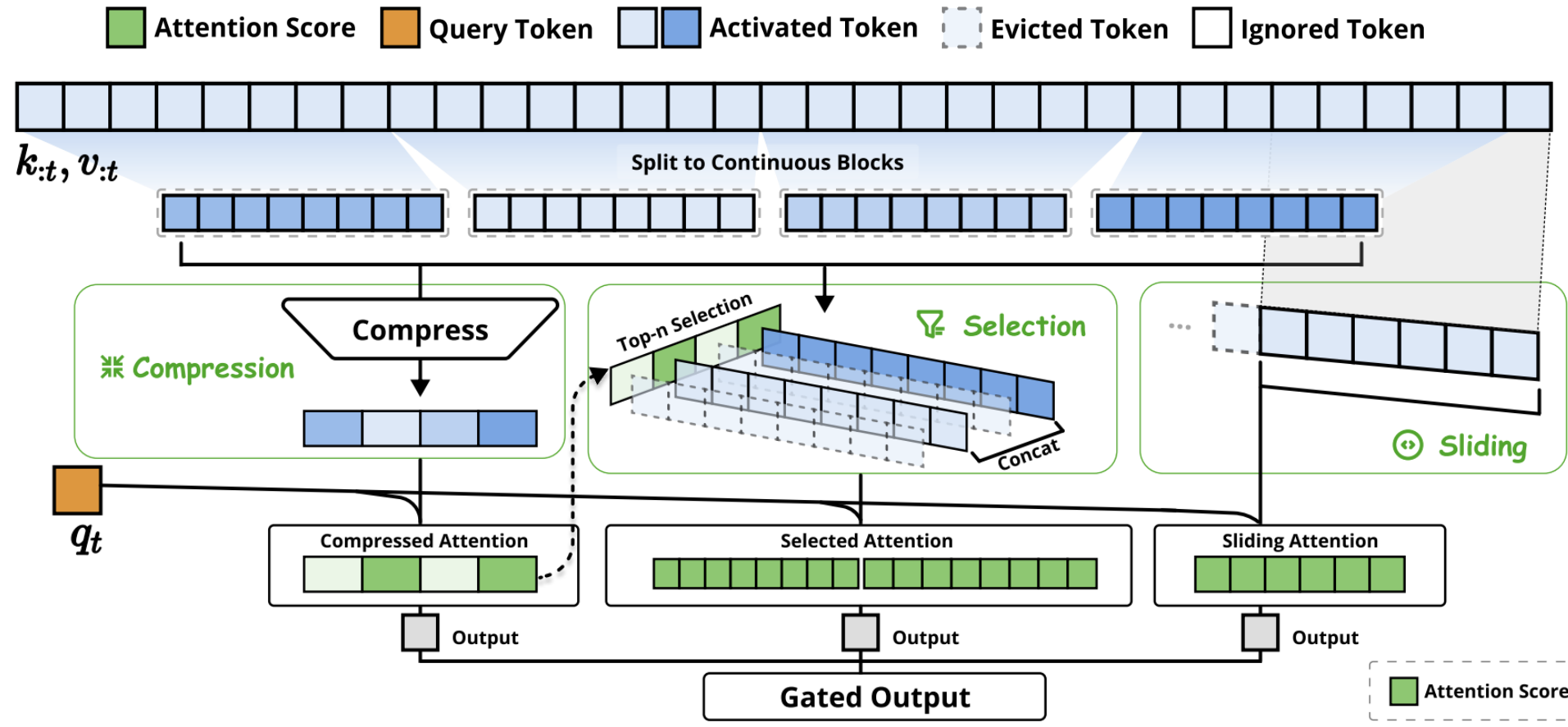


[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

Native Sparse Attention (ACL'25 from DeepSeek)



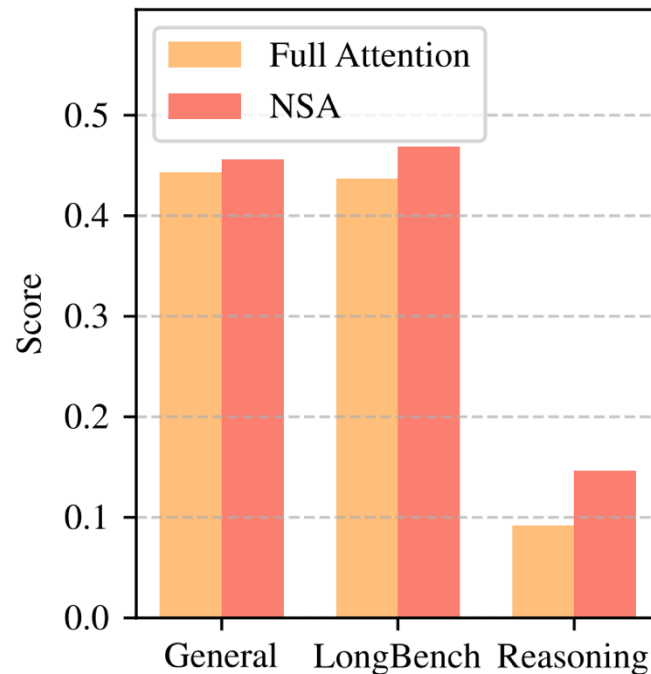
- Key insight: we can summarize long-context input, identify and focus on key words



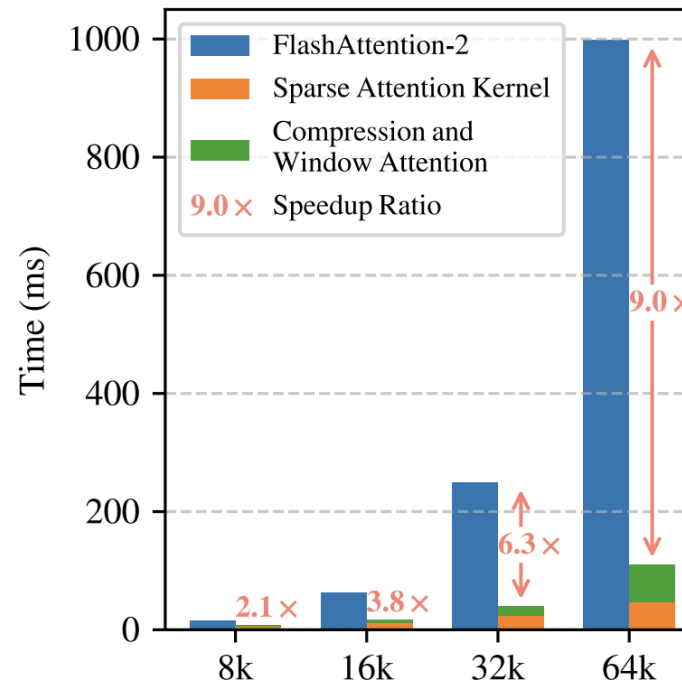
[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

- Better quality, compute, and memory footprint

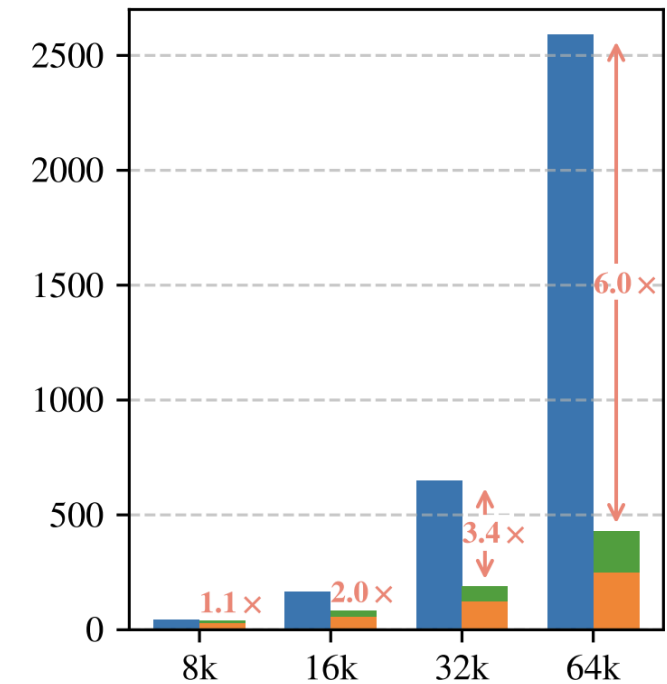
Performance on Benchmarks



Forward Time Comparison



Backward Time Comparison



Input length

[1] Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention, ACL, 2025

- **Transformers Deep Dive**

- Performance Analysis (e.g., arithmetic intensity)
 - Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot d$
- MLP layer is more computation-intensive than attention
- Multi-head & Multi-query Attention
 - Parallel execution of heads
- Native Sparse Attention (ACL'25 Best Paper Award)
 - Summarize, focus on key tokens and neighboring tokens

- **Transformers Deep Dive**

- Performance Analysis (e.g., arithmetic intensity)
 - Attention Compute: $(4 \cdot h + 3) \cdot N^2$; Memory IO: $8 \cdot N^2 + 8 \cdot N \cdot d$
- MLP layer is more computation-intensive than attention
- Multi-head & Multi-query Attention
 - Parallel execution of heads
- Native Sparse Attention (ACL'25 Best Paper Award)
 - Summarize, focus on key tokens and neighboring tokens

- **Paper summary by Feb 4 on Canvas**

- **Each summary (4 paragraphs) must follow the template**

- What is the problem and why is it important?
- The main proposed idea(s). A summary of your understanding of different components of the proposed technique, e.g., the purpose of critical designs.
- What are the technical drawbacks/limitations
- How will you improve them?

Grading criteria, each summary has 8 points in total. For each item here, you get:

- 2: The summary item demonstrates a clear understanding of the paper.
- 1: The summary item misses the point of the paper.
- 0: The summary item is missing.